Tela Soma – Documentação do desenvolvimento (AOSP Android 14)

Oct 25, 2026 · @Pedro Sadarc

Resumo e status atual

A Tela Soma é um novo modo de tela de bloqueio para o AOSP Android 14 (`android-14.0.0_r27`): o usuário vê dois números de 0 a 9 e desbloqueia digitando a soma. Ela já funciona de ponta a ponta no emulador e já aparece como opção em Configurações → Bloqueio de tela.

Onde paramos: Etapa 7 (persistência com `Settings.Secure`), **Passo 5 (compilar e testar)**. O Passo 1 foi concluído e salvo. Os Passos 2 e 3 também foram concluídos. Ordem combinada: testar o fluxo principal (Passo 5) e depois fazer o Passo 4, que é só visual. Nada foi compilado depois do Passo 1.

Etapa	Descrição	Situação
1	Reverter tentativa anterior e criar branch `tela-soma`	Concluída
2	Validar ciclo de build e envio ao emulador	Concluída
3	Interruptor e novo `SecurityMode.Math`	Concluída
4	Strings, layout, View e Controller da Tela Soma	Concluída
5	Integração com o Keyguard (Factory, ViewFlipper, desbloqueio, estado seguro)	Concluída
6	Teste da Tela Soma no emulador	Concluída, funcionou
Atividade 1 do Settings	Nome, ícone e descrição na lista de Bloqueio de tela	Concluída, funcionou
7	Persistência: trocar a propriedade `debug.` por `Settings.Secure`	Em andamento (Passos 1 a 3 de 5 concluídos)

Próximos passos, em ordem:

1. Confirmar o Passo 2 no `KeyguardStateControllerImpl` (detalhes na seção da Etapa 7).
2. Passo 3: o Settings grava `lockscreen_math_challenge_enabled` quando o usuário escolhe uma opção.
3. Passo 4: o Settings mostra “Tela Soma” como bloqueio atual.
4. Compilar `SystemUI` e `Settings`, enviar e executar o roteiro de testes.
5. Confirmar que os commits do SystemUI e do Settings foram feitos (não houve confirmação na conversa).

Atenção: depois do Passo 1, a propriedade `debug.sysui.math_lock` deixou de ter efeito. Até o Passo 3 ficar pronto, a Tela Soma só é ligada por adb: `adb shell settings put secure lockscreen_math_challenge_enabled 1`.

Arquitetura e decisões de design

A Tela Soma foi implementada como um novo `SecurityMode` dentro do Keyguard do SystemUI, sem criar uma credencial real no `LockSettingsService`. Ela substitui apenas o modo sem credencial (Deslizar).

Fluxo de execução, do Settings até o desbloqueio:

1. O usuário escolhe “Tela Soma” em Configurações → Bloqueio de tela (`ChooseLockGeneric`).
2. O Settings grava `lockscreen_math_challenge_enabled = 1` em `Settings.Secure` (Passo 3, pendente; hoje a chave é gravada por adb).
3. O `KeyguardMathLockConfig` (SystemUI) lê essa chave e avisa os interessados quando ela muda.
4. O `KeyguardStateControllerImpl` e o `KeyguardViewMediator` passam a considerar o aparelho “seguro”, então deslizar abre o bouncer em vez de desbloquear.
5. O `KeyguardSecurityModel` devolve `SecurityMode.Math` quando não há credencial e a chave está ligada.
6. O `KeyguardSecurityViewFlipperController` carrega o layout `keyguard_math_view.xml`.
7. A Factory do `KeyguardInputViewController` cria o `KeyguardMathViewController` para a `KeyguardMathView`.
8. O Controller sorteia os números, valida a resposta e chama `dismiss(true, usuário, Math)`.
9. O `KeyguardSecurityContainerController` aceita o modo `Math` e conclui o desbloqueio.

Decisões de design:

- **Não é credencial real.** Criar uma credencial exigiria mexer em `LockSettingsService`, Gatekeeper e criptografia de armazenamento (FBE). A Tela Soma é uma demonstração de customização do Keyguard, não um mecanismo de segurança: os números ficam visíveis e qualquer pessoa resolve a conta.
- **Credencial real sempre vence.** Se houver PIN, padrão ou senha, o `KeyguardSecurityModel` ignora a Tela Soma. Assim, a mudança nunca enfraquece um bloqueio existente.
- **`primaryAuth = false` no desbloqueio.** O Android não trata o desbloqueio como autenticação forte, que reabilitaria biometria ou renovaria Smart Lock.
- **Sem `reportUnlockAttempt`.** Erros na soma não entram no contador de tentativas falhas de credenciais reais.
- **Novos números a cada erro.** Impede tentar 0 a 18 em sequência.
- **Mapeamento para `Pin` no `AuthenticationRepository`.** Mapear para `None` faria partes do SystemUI acharem que o aparelho não tem bloqueio.
- **Fonte única da verdade.** A regra "Tela Soma torna o aparelho seguro" fica no `KeyguardStateControllerImpl`; o `KeyguardViewMediator` apenas consulta.
- **Padrão View + Controller e injeção via Dagger,** como no restante do Keyguard. O `Random` é injetável por um construtor `@VisibleForTesting`.
- **`when` exaustivo em Kotlin, sem `else`.** O compilador continua avisando se outro modo for adicionado no futuro.

Limitação conhecida: no `KeyguardViewMediator`, a Tela Soma é considerada apenas para o usuário atual. No emulador, com um usuário, isso não tem efeito.

Ambiente e ciclo de desenvolvimento

O projeto usa AOSP `android-14.0.0_r27`, alvo `sdk_phone_x86_64-userdebug`, em `~/aosp`. Os dois projetos alterados têm o branch de trabalho `tela-soma`.

Item	Valor
Versão	`android-14.0.0_r27`
Alvo do `lunch`	`sdk_phone_x86_64-userdebug`
Saída da build	`~/aosp/out/target/product/emulator_x86_64`
Projetos alterados	`frameworks/base` (SystemUI) e `packages/apps/Settings`

Item	Valor
Branch	`tela-soma` (criado com `repo start`)
Onde ficam os APKs	`/system_ext/priv-app/SystemUI` e `/system_ext/priv-app/Settings`

Preparar o terminal (sempre no mesmo terminal):

```
cd ~/aosp
source build/envsetup.sh
lunch sdk_phone_x86_64-userdebug
```

Iniciar o emulador com sistema gravável:

```
emulator -writable-system
```

Na primeira vez (ou após `-wipe-data`): `adb root`, `adb remount`, `adb reboot`, `adb wait-for-device`, `adb root`, `adb remount`.

Ciclo rápido de compilação e envio:

```
m SystemUI 2>&1 | tee ~/build.log      # ou: m Settings
grep -E "error:|^e: " ~/build.log
adb root && adb remount
adb sync system_ext
adb shell pkill -f com.android.systemui      # para o SystemUI
adb shell am force-stop com.android.settings # para o Settings
```

Ligar e desligar a Tela Soma por adb (a partir da Etapa 7):

```
adb shell settings put secure lockscreen_math_challenge_enabled 1
adb shell settings put secure lockscreen_math_challenge_enabled 0
```

Até a Etapa 6, o interruptor era `adb shell setprop debug.sysui.math_lock true`, que se perdia a cada reinício.

SystemUI: arquivos criados

Foram criados 5 arquivos em `~/aosp/frameworks/base/packages/SystemUI`. O conteúdo abaixo é a versão atual de cada um.

Arquivo	Função
`res-keyguard/values/strings_math_lock.xml`	Textos da Tela Soma no bouncer

Arquivo	Função
`res-keyguard/layout/keyguard_math_view.xml`	Layout da Tela Soma
`src/com/android/keyguard/KeyguardMathView.java`	View: só apresentação
`src/com/android/keyguard/KeyguardMathViewController.java`	Controller: sorteio, validação e desbloqueio
`src/com/android/keyguard/KeyguardMathLockConfig.java`	Interruptor: lê `Settings.Secure` e notifica mudanças

strings_math_lock.xml

`~/aosp/frameworks/base/packages/SystemUI/res-keyguard/values/strings_math_lock.xml`

Por quê: textos em recurso permitem tradução e seguem a regra do AOSP. O `xliff:g` marca para tradutores os trechos `%1$d` e `%2$d`, que são preenchidos pelo código.

```
<?xml version="1.0" encoding="utf-8"?>
<!-- Textos da Tela Soma (desafio matemático no keyguard) -->
<resources xmlns:xliff="urn:oasis:names:tc:xliff:document:1.2">
    <string name="keyguard_math_title">Tela Soma</string>
    <string name="keyguard_math_instruction">Digite a soma dos números para
desbloquear</string>
    <string name="keyguard_math_challenge"><xliff:g id="first"
example="3">%1$d</xliff:g> + <xliff:g id="second" example="5">%2$d</xliff:g>
</string>
    <string name="keyguard_math_answer_hint">Resultado</string>
    <string name="keyguard_math_confirm">Desbloquear</string>
    <string name="keyguard_math_wrong">Resposta errada. Tente com os novos
números.</string>
</resources>
```

keyguard_math_view.xml

`~/aosp/frameworks/base/packages/SystemUI/res-keyguard/layout/keyguard_math_view.xml`

Por quê: define a tela. A raiz é a `KeyguardMathView`. O `include` da área de mensagens é obrigatório, porque a classe base usa o `mMessageAreaController` sem checar nulo. A instrução aparece nessa área nativa (via `getInitialMessageResId`), por isso não há `TextView` de instrução.

```
<?xml version="1.0" encoding="utf-8"?>
<!-- Tela Soma: o usuário digita a soma de dois números de 0 a 9 -->
<com.android.keyguard.KeyguardMathView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/keyguard_math_view"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_gravity="center_horizontal|bottom"
    android:gravity="center"
    android:orientation="vertical"
    android:padding="24dp">

    <include layout="@layout/keyguard_bouncer_message_area" />

    <TextView
        android:id="@+id/math_challenge"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:textColor="?android:attr/textColorPrimary"
        android:textSize="56sp"
        android:textStyle="bold" />

    <EditText
        android:id="@+id/math_answer"
        android:layout_width="160dp"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:gravity="center"
        android:hint="@string/keyguard_math_answer_hint"
        android:imeOptions="actionDone|flagNoExtractUi"
        android:importantForAutofill="no"
        android:inputType="number"
        android:maxLength="2"
        android:textColor="?android:attr/textColorPrimary"
        android:textSize="24sp" />

    <TextView
        android:id="@+id/math_error"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:gravity="center"
        android:text="@string/keyguard_math_wrong"
        android:textColor="?android:attr/colorError"
        android:visibility="invisible" />
```

```
    <Button
        android:id="@+id/math_confirm"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:text="@string/keyguard_math_confirm" />

</com.android.keyguard.KeyguardMathView>
```

KeyguardMathView.java

`~/aosp/frameworks/base/packages/SystemUI/src/com/android/keyguard/KeyguardMathView.java`

Por quê: a View herda de `KeyguardInputView`, a base de todas as telas do bouncer. Ela não tem lógica, só mostra o que o Controller manda. O import do `R` é `com.android.systemui.R` nesta versão.

```
/*
 * Copyright (C) 2026 The Android Open Source Project
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *      http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */

package com.android.keyguard;

import android.content.Context;
import android.util.AttributeSet;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;

import com.android.systemui.R;
```

```
/**
 * View da Tela Soma. Responsável apenas pela apresentação;
 * toda a lógica fica em {@link KeyguardMathViewController}.
 */
public class KeyguardMathView extends KeyguardInputView {

    private TextView mChallengeView;
    private EditText mAnswerView;
    private TextView mErrorView;
    private Button mConfirmButton;

    public KeyguardMathView(Context context) {
        this(context, null);
    }

    public KeyguardMathView(Context context, AttributeSet attrs) {
        super(context, attrs);
    }

    @Override
    protected void onFinishInflate() {
        super.onFinishInflate();
        mChallengeView = findViewById(R.id.math_challenge);
        mAnswerView = findViewById(R.id.math_answer);
        mErrorView = findViewById(R.id.math_error);
        mConfirmButton = findViewById(R.id.math_confirm);
    }

    /** Mostra um novo desafio e limpa a resposta anterior. */
    void showChallenge(int first, int second) {
        mChallengeView.setText(
                getContext().getString(R.string.keyguard_math_challenge,
first, second));
        mAnswerView.setText("");
    }

    /** Mostra ou esconde a mensagem de resposta errada. */
    void setErrorVisible(boolean visible) {
        mErrorView.setVisibility(visible ? View.VISIBLE : View.INVISIBLE);
    }

    String getAnswerText() {
        return mAnswerView.getText().toString().trim();
    }

    EditText getAnswerView() {
```

```
        return mAnswerView;
    }

    Button getConfirmButton() {
        return mConfirmButton;
    }

    @Override
    public CharSequence getTitle() {
        return getContext().getString(R.string.keyguard_math_title);
    }
}
```

KeyguardMathViewController.java

`~/aosp/frameworks/base/packages/SystemUI/src/com/android/keyguard/KeyguardMathViewController.java`

Por quê: concentra a lógica. Sorteia dois números de 0 a 9, valida a resposta, gera novos números a cada erro e, ao acertar, chama `dismiss`. O construtor segue exatamente a assinatura da classe base desta versão (6 parâmetros, incluindo `FeatureFlags`). `getInitialMessageResId()` e `needsInput()` são obrigatórios.

```
/*
 * Copyright (C) 2026 The Android Open Source Project
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *      http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */

package com.android.keyguard;

import android.annotation.Nullable;
import android.view.KeyEvent;
import android.view.WindowInsets;
import android.view.inputmethod.EditorInfo;
import android.widget.EditText;
```

```
import android.widget.TextView;

import androidx.annotation.VisibleForTesting;

import com.android.keyguard.KeyguardSecurityModel.SecurityMode;
import com.android.systemui.R;
import com.android.systemui.flags.FeatureFlags;

import java.util.Random;

/**
 * Controller da Tela Soma: sorteia dois números de 0 a 9 e desbloqueia
 * o dispositivo quando o usuário digita a soma correta.
 */
public class KeyguardMathViewController
        extends KeyguardInputViewController<KeyguardMathView> {

    private static final int MAX_OPERAND = 9;

    private final Random mRandom;
    private int mExpectedAnswer;

    private final TextView.OnEditorActionListener mOnEditorActionListener =
            (v, actionId, event) -> {
                final boolean isImeDone = actionId ==
EditorInfo.IME_ACTION_DONE;
                final boolean isEnterKey = event != null
                        && event.getKeyCode() == KeyEvent.KEYCODE_ENTER
                        && event.getAction() == KeyEvent.ACTION_DOWN;
                if (isImeDone || isEnterKey) {
                    verifyAnswer();
                    return true;
                }
                return false;
            };

    KeyguardMathViewController(KeyguardMathView view,
            SecurityMode securityMode,
            KeyguardSecurityCallback keyguardSecurityCallback,
            EmergencyButtonController emergencyButtonController,
            @Nullable KeyguardMessageAreaController.Factory
messageAreaControllerFactory,
            FeatureFlags featureFlags) {
        this(view, securityMode, keyguardSecurityCallback,
emergencyButtonController,
                messageAreaControllerFactory, featureFlags, new Random());
```

```
    }

    @VisibleForTesting
    KeyguardMathViewController(KeyguardMathView view,
            SecurityMode securityMode,
            KeyguardSecurityCallback keyguardSecurityCallback,
            EmergencyButtonController emergencyButtonController,
            @Nullable KeyguardMessageAreaController.Factory
messageAreaControllerFactory,
            FeatureFlags featureFlags,
            Random random) {
        super(view, securityMode, keyguardSecurityCallback,
emergencyButtonController,
                messageAreaControllerFactory, featureFlags);
        mRandom = random;
    }

    @Override
    protected void onViewAttached() {
        super.onViewAttached();
        mView.getConfirmButton().setOnClickListener(v -> verifyAnswer());

mView.getAnswerView().setOnEditorActionListener(mOnEditorActionListener);
    }

    @Override
    protected void onViewDetached() {
        super.onViewDetached();
        mView.getConfirmButton().setOnClickListener(null);
        mView.getAnswerView().setOnEditorActionListener(null);
    }

    @Override
    public void onResume(int reason) {
        super.onResume(reason);
        generateChallenge();
        showKeyboard();
    }

    @Override
    public void reset() {
        super.reset();
        generateChallenge();
    }

    @Override
```

```
    public boolean needsInput() {
        // Precisa do teclado para digitar a resposta.
        return true;
    }

    @Override
    protected int getInitialMessageResId() {
        return R.string.keyguard_math_instruction;
    }

    private void generateChallenge() {
        final int first = mRandom.nextInt(MAX_OPERAND + 1);
        final int second = mRandom.nextInt(MAX_OPERAND + 1);
        mExpectedAnswer = first + second;
        mView.showChallenge(first, second);
        mView.setErrorVisible(false);
    }

    private void verifyAnswer() {
        // Mantém a tela ligada enquanto o usuário interage.
        getKeyguardSecurityCallback().userActivity();

        final String answer = mView.getAnswerText();
        if (answer.isEmpty()) {
            return;
        }

        final int value;
        try {
            value = Integer.parseInt(answer);
        } catch (NumberFormatException e) {
            onWrongAnswer();
            return;
        }

        if (value == mExpectedAnswer) {
            getKeyguardSecurityCallback().dismiss(
                    /* securityVerified= */ true,
                    KeyguardUpdateMonitor.getCurrentUser(),
                    getSecurityMode());
        } else {
            onWrongAnswer();
        }
    }

    private void onWrongAnswer() {
```

```
        // Gera novos números a cada erro para impedir "chutes" sequenciais.
        generateChallenge();
        mView.setErrorVisible(true);
    }

    private void showKeyboard() {
        final EditText answerView = mView.getAnswerView();
        mView.post(() -> {
            if (mView.isShown()) {
                answerView.requestFocus();

answerView.getWindowInsetsController().show(WindowInsets.Type.ime());
            }
        });
    }
}
```

KeyguardMathLockConfig.java

`~/aosp/frameworks/base/packages/SystemUI/src/com/android/keyguard/KeyguardMathLockConfig.java`

Por quê: ponto único de configuração. Criado na Etapa 3 lendo a propriedade `debug.sysui.math_lock`. Reescrito no Passo 1 da Etapa 7 para ler `Settings.Secure` por usuário (sobrevive a reinício) e notificar listeners via `ContentObserver`. Usa o `SecureSettings` do SystemUI, que pode ser substituído em testes.

```
/*
 * Copyright (C) 2026 The Android Open Source Project
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *      http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */

package com.android.keyguard;

import android.database.ContentObserver;
```

```
import android.os.Handler;
import android.os.UserHandle;

import com.android.systemui.dagger.SysUISingleton;
import com.android.systemui.dagger.qualifiers.Main;
import com.android.systemui.util.settings.SecureSettings;

import java.util.concurrent.CopyOnWriteArrayList;

import javax.inject.Inject;

/**
 * Ponto único de configuração da "Tela Soma" (desafio matemático no
keyguard).
 *
 * O valor é gravado pelo app de Configurações em Settings.Secure, por
usuário, e
 * sobrevive a reinícios. Significado: quando o usuário NÃO tem credencial
(PIN,
 * padrão ou senha), o desafio matemático substitui o simples deslizar.
 *
 * Para testes via adb:
 *   adb shell settings put secure lockscreen_math_challenge_enabled 1
 *   adb shell settings put secure lockscreen_math_challenge_enabled 0
 */
@SysUISingleton
public class KeyguardMathLockConfig {

    /** Chave em Settings.Secure. Precisa ser idêntica à usada no app de
Configurações. */
    public static final String SETTING_MATH_LOCK_ENABLED =
"lockscreen_math_challenge_enabled";

    /** Notificado quando a configuração muda, para o Keyguard se atualizar
na hora. */
    public interface Listener {
        void onMathLockConfigChanged();
    }

    private final SecureSettings mSecureSettings;
    private final CopyOnWriteArrayList<Listener> mListeners = new
CopyOnWriteArrayList<>();

    @Inject
    public KeyguardMathLockConfig(SecureSettings secureSettings, @Main
Handler mainHandler) {
```

```
        mSecureSettings = secureSettings;

        final ContentObserver observer = new ContentObserver(mainHandler) {
            @Override
            public void onChange(boolean selfChange) {
                for (Listener listener : mListeners) {
                    listener.onMathLockConfigChanged();
                }
            }
        };
        // USER_ALL: observa a chave de todos os usuários, para funcionar
também
        // depois de uma troca de usuário.
        mSecureSettings.registerContentObserverForUser(
                SETTING_MATH_LOCK_ENABLED, observer, UserHandle.USER_ALL);
    }

    /** Retorna true se a Tela Soma está ativa para o usuário atual. */
    public boolean isEnabled() {
        return isEnabledForUser(KeyguardUpdateMonitor.getCurrentUser());
    }

    /** Retorna true se a Tela Soma está ativa para o usuário informado. */
    public boolean isEnabledForUser(int userId) {
        return mSecureSettings.getIntForUser(SETTING_MATH_LOCK_ENABLED, 0,
userId) == 1;
    }

    public void addListener(Listener listener) {
        mListeners.addIfAbsent(listener);
    }

    public void removeListener(Listener listener) {
        mListeners.remove(listener);
    }
}
```

SystemUI: arquivos modificados

Foram modificados 7 arquivos existentes. Abaixo estão apenas os trechos inseridos ou alterados; o restante de cada arquivo é o original do AOSP.

Arquivo	O que mudou
`KeyguardSecurityModel.java`	Novo modo `Math` e regra de escolha
`AuthenticationRepository.kt`	`Math` mapeado para `Pin`
`KeyguardInputViewController.java`	Factory cria o Controller da Tela Soma
`KeyguardSecurityViewFlipperController.java`	Layout e ID da Tela Soma
`KeyguardSecurityContainerController.java`	Aceita o desbloqueio do modo `Math`
`KeyguardStateControllerImpl.java`	Tela Soma torna o aparelho “seguro”
`KeyguardViewMediator.java`	`isSecure` consulta o estado acima

KeyguardSecurityModel.java

`~/aosp/frameworks/base/packages/SystemUI/src/com/android/keyguard/KeyguardSecurityModel.java`

Por quê: é quem decide qual tela de segurança mostrar. Recebeu o novo valor do enum e passa a devolvê-lo quando não há credencial e a Tela Soma está ligada.

```
// 1. Novo valor no enum SecurityMode, depois de SimPuk:
        SimPuk, // Unlock by entering a sim puk
        Math // Desafio matemático: digitar a soma de dois números (Tela
Soma)

// 2. Novo campo:
    private final KeyguardMathLockConfig mMathLockConfig;

// 3. Novo parâmetro no construtor e atribuição:
    KeyguardSecurityModel(@Main Resources resources, LockPatternUtils
lockPatternUtils,
            KeyguardUpdateMonitor keyguardUpdateMonitor,
            KeyguardMathLockConfig mathLockConfig) {
        ...
        mMathLockConfig = mathLockConfig;
    }

// 4. No switch de getSecurityMode(int userId):
            case DevicePolicyManager.PASSWORD_QUALITY_UNSPECIFIED:
                // A Tela Soma só substitui o modo sem credencial. Se houver
PIN,
```

```
                // padrão ou senha real, o mecanismo original continua
valendo.
                return mMathLockConfig.isEnabled() ? SecurityMode.Math :
SecurityMode.None;
```

AuthenticationRepository.kt

`~/aosp/frameworks/base/packages/SystemUI/src/com/android/systemui/authentication/data/repository/AuthenticationRepository.kt`

Por quê: o `when` do método `blockingAuthenticationMethodInternal` é exaustivo, e a build falhou ao adicionar `Math` ao enum. O modo foi mapeado para `Pin` porque exige entrada no bouncer. Usa o modelo da camada de dados (`authentication.data.model`).

```
            KeyguardSecurityModel.SecurityMode.SimPuk ->
AuthenticationMethodModel.Pin
            // Tela Soma: tratada como Pin porque exige entrada numérica no
bouncer.
            // Mapear para None faria o SystemUI considerar o aparelho sem
bloqueio.
            KeyguardSecurityModel.SecurityMode.Math ->
AuthenticationMethodModel.Pin
            KeyguardSecurityModel.SecurityMode.Password ->
AuthenticationMethodModel.Password
```

KeyguardInputViewController.java

`~/aosp/frameworks/base/packages/SystemUI/src/com/android/keyguard/KeyguardInputViewController.java`

Por quê: a classe interna `Factory` cria o Controller certo para cada View inflada. Sem este bloco, o SystemUI lançaria `RuntimeException("Unable to find controller ...")`.

```
// No método Factory.create(...), depois do bloco de KeyguardSimPukView:
            } else if (keyguardInputView instanceof KeyguardMathView) {
                // Tela Soma: desafio matemático (ver
KeyguardMathLockConfig).
                return new KeyguardMathViewController((KeyguardMathView)
keyguardInputView,
                        securityMode, keyguardSecurityCallback,
emergencyButtonController,
                        mMessageAreaControllerFactory, mFeatureFlags);
            }
```

KeyguardSecurityViewFlipperController.java

`~/aosp/frameworks/base/packages/SystemUI/src/com/android/keyguard/KeyguardSecurityViewFlipperController.java`

Por quê: decide qual layout inflar para cada modo. Os dois `switch` têm `default: return 0`, então o compilador não avisa; sem estas linhas, a Tela Soma simplesmente não apareceria.

```
// Em getLayoutIdFor(SecurityMode):
            case Math: return R.layout.keyguard_math_view; // Tela Soma

// Em getKeyguardInputViewId(SecurityMode):
            case Math: return R.id.keyguard_math_view; // Tela Soma
```

KeyguardSecurityContainerController.java

`~/aosp/frameworks/base/packages/SystemUI/src/com/android/keyguard/KeyguardSecurityContainerController.java`

Por quê: no método `showNextSecurityScreenOrFinish`, modos desconhecidos caíam em "Bad security screen, fail safe" e o aparelho não desbloqueava. `primaryAuth = false` porque não há credencial real verificada.

```
// No switch (getCurrentSecurityMode()) dentro de else if (authenticated),
entre PIN e SimPin:
                case Math:
                    // Tela Soma: desbloqueia, mas NÃO conta como
autenticação primária,
                    // pois não há credencial real verificada pelo
LockSettingsService.
                    primaryAuth = false;
                    finish = true;
                    eventSubtype = BOUNCER_DISMISS_NONE_SECURITY;
                    uiEvent = BouncerUiEvent.BOUNCER_DISMISS_NONE_SECURITY;
                    break;
```

KeyguardStateControllerImpl.java

`~/aosp/frameworks/base/packages/SystemUI/src/com/android/systemui/statusbar/policy/KeyguardStateControllerImpl.java`

Por quê: mantém o estado consultado por todo o SystemUI. Sem esta mudança, o sistema acharia que não há bloqueio e o deslizar desbloquearia direto. A atribuição do campo precisa vir antes da chamada `update(true)` do construtor, senão ocorre

`NullPointerException`.

```
// 1. Import:
import com.android.keyguard.KeyguardMathLockConfig;

// 2. Campo, abaixo de mLockPatternUtils:
    private final KeyguardMathLockConfig mMathLockConfig;

// 3. Construtor: novo último parâmetro e atribuição depois de
mLockPatternUtils:
            FeatureFlags featureFlags,
            KeyguardMathLockConfig mathLockConfig) {
        ...
        mMathLockConfig = mathLockConfig;

// 4. No método update(boolean updateAlways):
        // Tela Soma: sem credencial real, o desafio matemático também exige
o bouncer.
        boolean secure = mLockPatternUtils.isSecure(user) ||
mMathLockConfig.isEnabled();
```

O Passo 2 da Etapa 7 altera este arquivo novamente (ver a seção da Etapa 7).

KeyguardViewMediator.java

`~/aosp/frameworks/base/packages/SystemUI/src/com/android/systemui/keyguard/KeyguardViewMediator.java`

Por quê: o “maestro” do Keyguard tem seu próprio `isSecure`. Ele já tinha acesso ao `KeyguardStateController`, então apenas o consulta, sem mexer no construtor nem no módulo Dagger.

```
    public boolean isSecure(int userId) {
        return mLockPatternUtils.isSecure(userId)
                || mUpdateMonitor.isSimPinSecure()
                || isMathLockActive(userId);
    }

    /**
     * Tela Soma: delega ao KeyguardStateController, que é a fonte única da
verdade
     * sobre o desafio matemático (ver KeyguardMathLockConfig).
     */
    private boolean isMathLockActive(int userId) {
        return userId == KeyguardUpdateMonitor.getCurrentUser()
                && mKeyguardStateController.isMethodSecure();
    }
```

Settings: arquivos criados e modificados

No app de Configurações (`~/aosp/packages/apps/Settings`) foram criados 2 arquivos e modificados 3. Resultado: a Tela Soma aparece em Configurações → Segurança e privacidade → Desbloqueio do dispositivo → Bloqueio de tela, logo abaixo de Deslizar. Tocá-la ainda não altera o bloqueio (Passo 3 da Etapa 7).

Arquivo	Tipo	Função
`res/values/strings_math_lock.xml`	Criado	Nome e descrição da opção
`res/drawable/ic_lock_math.xml`	Criado	Ícone “+”
`res/xml/security_settings_picker.xml`	Modificado	Item na lista de bloqueio
`src/com/android/settings/password/ScreenLockType.java`	Modificado	Tipo `MATH`
`src/com/android/settings/password/ChooseLockGenericController.java`	Modificado	Visibilidade e título

strings_math_lock.xml (criado)

`~/aosp/packages/apps/Settings/res/values/strings_math_lock.xml`

Por quê: arquivo separado do `strings.xml` principal evita conflitos em atualizações do AOSP e deixa a customização isolada.

```
<?xml version="1.0" encoding="utf-8"?>
<!-- Textos da Tela Soma na tela de escolha do bloqueio -->
<resources>
    <!-- Nome da opção na lista de Bloqueio de tela -->
    <string name="unlock_set_unlock_math_title">Tela Soma</string>
    <!-- Descrição curta exibida abaixo do nome -->
    <string name="unlock_set_unlock_math_summary">Digite a soma de dois
números para desbloquear</string>
</resources>
```

ic_lock_math.xml (criado)

`~/aosp/packages/apps/Settings/res/drawable/ic_lock_math.xml`

Por quê: ícone vetorial de "+". O `tint` com `colorControlNormal` segue o tema, como os ícones das outras opções.

```
<?xml version="1.0" encoding="utf-8"?>
<!-- Ícone da Tela Soma: sinal de mais -->
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="24dp"
    android:height="24dp"
    android:viewportWidth="24"
    android:viewportHeight="24"
    android:tint="?android:attr/colorControlNormal">
    <path
        android:fillColor="@android:color/white"
        android:pathData="M11,5h2v6h6v2h-6v6h-2v-6H5v-2h6z" />
</vector>
```

security_settings_picker.xml (modificado)

`~/aosp/packages/apps/Settings/res/xml/security_settings_picker.xml`

Por quê: é a lista exibida em Bloqueio de tela. O item foi inserido depois de `unlock_set_none` (Deslizar). A chave `unlock_set_math` precisa ser idêntica à do `ScreenLockType`.

```
    <!-- Tela Soma: desafio matemático no lugar do deslizar -->
    <com.android.settingslib.RestrictedPreference
            android:key="unlock_set_math"
            android:title="@string/unlock_set_unlock_math_title"
            android:summary="@string/unlock_set_unlock_math_summary"
            android:icon="@drawable/ic_lock_math"
            android:persistent="false"/>
```

ScreenLockType.java (modificado)

`~/aosp/packages/apps/Settings/src/com/android/settings/password/ScreenLockType.java`

Por quê: enum com os tipos de bloqueio que o Settings conhece. `MATH` usa a mesma qualidade do Deslizar porque não há credencial real. O `fromQuality` não foi alterado.

```
// Inserido entre SWIPE e PATTERN:
    // Tela Soma: mesma qualidade do SWIPE, pois não há credencial real.
    MATH(
            DevicePolicyManager.PASSWORD_QUALITY_UNSPECIFIED,
            "unlock_set_math"),
```

ChooseLockGenericController.java (modificado)

`~/aosp/packages/apps/Settings/src/com/android/settings/password/ChooseLockGenericController.java`

Por quê: decide quais opções ficam visíveis e qual título cada uma tem. Os dois `switch` têm `return` genérico depois, então o compilador não avisaria. A Tela Soma segue as regras do Deslizar: escondida em perfis de trabalho e quando o chamador exige bloqueio seguro (ex.: cadastro de digital). O `isScreenLockEnabled` não mudou: uma política que exija senha já desabilita a opção.

```
// Em isScreenLockVisible(ScreenLockType), depois do case SWIPE:
            case MATH:
                // Tela Soma: não é uma credencial real, então segue as
mesmas regras de
                // visibilidade dos tipos inseguros (escondida quando o
chamador pede
                // apenas bloqueios seguros e em perfis de trabalho).
                return !mHideInsecureScreenLockTypes
                    && !managedProfile;

// Em getTitle(ScreenLockType), depois do case SWIPE:
            case MATH:
                return
mContext.getText(R.string.unlock_set_unlock_math_title);
```

Etapa 7: persistência (em andamento)

Objetivo do professor: manter o bloqueio durante mudanças de estado e de método. Atividades: trocar um método existente pelo novo e voltar a um método existente; verificar reinício e apagar/acender a tela.

A solução troca a propriedade `debug.sysui.math_lock` (perdida no reinício, só alterável por adb) pela chave `lockscreen_math_challenge_enabled` em `Settings.Secure` (gravada em disco, por usuário, alterável pelo Settings). Regra: a chave significa “quando não houver credencial, use a Tela Soma em vez do deslizar”; um PIN sempre vence.

Passo	Descrição	Arquivo	Situação
1	SystemUI lê `Settings.Secure`	`KeyguardMathLockConfig.java`	Concluído e salvo
2	SystemUI reage na hora à mudança	`KeyguardStateControllerImpl.java`	Concluído e conferido
3	Settings grava a chave ao escolher uma opção	`ChooseLockGeneric.java`	Concluído e conferido
4	Settings mostra “Tela Soma” como bloqueio atual	MathLockUtils.java + tela de Segurança	Pendente
5	Compilar, enviar e executar o roteiro de testes	—	Pendente

Passo 2 em detalhe (para retomar)

Arquivo: `~/aosp/frameworks/base/packages/SystemUI/src/com/android/systemui/statusbar/policy/KeyguardStateControllerImpl.java`

Por quê: hoje o estado só é recalculado ao iniciar o SystemUI ou em eventos do Keyguard, por isso era preciso `pkill`. Com o listener, ele recalcula assim que a chave muda.

Alteração 1, no construtor (troca a linha `mMathLockConfig = mathLockConfig;` por):

```
        mMathLockConfig = mathLockConfig;
        // Tela Soma: recalcula o estado do Keyguard assim que a configuração
mudar.
        mMathLockConfig.addListener(() -> update(false /* updateAlways */));
```

Alteração 2, no método `update` (avalia para o mesmo usuário do resto do método):

```
        boolean secure = mLockPatternUtils.isSecure(user) ||
mMathLockConfig.isEnabledForUser(user);
```

Verificação: o comando abaixo deve listar 6 linhas (import, campo, parâmetro, atribuição, `addListener`, `isEnabledForUser`).

```
grep -n "MathLockConfig"
~/aosp/frameworks/base/packages/SystemUI/src/com/android/systemui/statusbar/p
olicy/KeyguardStateControllerImpl.java
```

Passo 3 em detalhe (concluído)

Arquivo criado:
`~/aosp/packages/apps/Settings/src/com/android/settings/password/MathLockUtils.java`

Por quê: concentra no Settings a leitura e gravação da chave, que também será usada no Passo 4. A constante deve ser idêntica à do SystemUI.

```
package com.android.settings.password;

import android.content.Context;
import android.provider.Settings;

public final class MathLockUtils {

    /** Idêntica a KeyguardMathLockConfig.SETTING_MATH_LOCK_ENABLED
(SystemUI). */
    public static final String SETTING_MATH_LOCK_ENABLED =
"lockscreen_math_challenge_enabled";

    private MathLockUtils() {
    }

    public static boolean isEnabled(Context context, int userId) {
        return Settings.Secure.getIntForUser(context.getContentResolver(),
                SETTING_MATH_LOCK_ENABLED, 0, userId) == 1;
    }

    public static void setEnabled(Context context, int userId, boolean
enabled) {
```

```
        Settings.Secure.putIntForUser(context.getContentResolver(),
                SETTING_MATH_LOCK_ENABLED, enabled ? 1 : 0, userId);
    }
}
```

Arquivo modificado: `~/aosp/packages/apps/Settings/src/com/android/settings/password/ChooseLockGeneric.java`

Por quê: o `setUnlockMethod` não conhecia `MATH` e registrava “Encountered unknown unlock method”. O `isUnlockMethodSecure` tratava a Tela Soma como segura, então o aviso de Proteção contra redefinição de fábrica (FRP) não apareceria ao sair de um PIN.

```
// isUnlockMethodSecure: MATH passa a ser considerado inseguro.
            return !(ScreenLockType.SWIPE.preferenceKey.equals(unlockMethod)
                    || ScreenLockType.NONE.preferenceKey.equals(unlockMethod)
                    ||
ScreenLockType.MATH.preferenceKey.equals(unlockMethod));

// setUnlockMethod: MATH entra no grupo de NONE e SWIPE (remove a
credencial).
                    case NONE:
                    case SWIPE:
                    case MATH:
                        updateUnlockMethodAndFinish(
                                lock.defaultQuality,
                                lock == ScreenLockType.NONE,
                                false /* chooseLockSkipped */);
                        // Tela Soma: liga para MATH e desliga para NONE e
SWIPE. Gravado
                        // depois da troca, para não ligar se ela falhar.
                        MathLockUtils.setEnabled(getContext(), mUserId,
                                lock == ScreenLockType.MATH);
                        return true;
                    case PATTERN:
                    case PIN:
                    case PASSWORD:
                    case MANAGED:
                        // inalterado: abre o cadastro da credencial, sem
mexer na chave
```

PIN, Padrão e Senha não alteram a chave: o usuário pode cancelar o cadastro, e o PIN sempre vence no SystemUI enquanto existir.

Plano para os Passos 3 e 4

- Passo 3: em `ChooseLockGeneric.java`, ao escolher `MATH`, remover a credencial atual (mesmo caminho do Deslizar) e gravar a chave com valor 1; ao escolher `SWIPE` ou `NONE`, gravar 0. A chave deve ser o mesmo texto da constante `KeyguardMathLockConfig.SETTING_MATH_LOCK_ENABLED`.
- Passo 4: o resumo do bloqueio atual (hoje “Deslizar”) deve mostrar “Tela Soma” quando a chave for 1 e não houver credencial.
- A chave não é declarada em `Settings.Secure` do framework, para evitar recompilar o `framework.jar`. SystemUI e Settings são apps do sistema e podem ler e gravar chaves fora da lista pública. Se surgir erro de permissão, revisar essa decisão.

Testes

Todos os testes até aqui foram manuais, no emulador `sdk_phone_x86_64-userdebug`, e passaram.

Teste	Etapa	Resultado
Log de marcador aparece após `m SystemUI` + `adb sync`	2	Passou
Build com `SecurityMode.Math` (Checkpoint A)	4	Passou após corrigir o `when` do `AuthenticationRepository`
Regressão com interruptor desligado: SystemUI sobe, bloquear/desbloquear normal, sem crash	5	Passou
Build da integração com o Keyguard (Checkpoint B)	5	Passou
Tela Soma aparece ao deslizar; erro gera novos números; acerto desbloqueia	6	Passou
Interruptor desligado volta ao deslizar normal	6	Passou
Opção “Tela Soma” aparece em Bloqueio de tela	Settings	Passou

Verificar crashes durante qualquer teste:

```
adb logcat -b crash
```

Roteiro para as atividades da Etapa 7

Executar depois dos Passos 2 a 4, com o bloqueio inicial em Deslizar.

- [ ] Escolher Tela Soma em Bloqueio de tela; apagar e acender a tela; deslizar mostra a Tela Soma.
- [ ] Trocar de Tela Soma para PIN; apagar e acender; o PIN é pedido, não a Tela Soma.
- [ ] Trocar de PIN para Tela Soma (o Settings pede o PIN antes); apagar e acender; a Tela Soma aparece.
- [ ] Trocar de Tela Soma para Deslizar; apagar e acender; deslizar desbloqueia direto.
- [ ] Com Tela Soma ativa, reiniciar (`adb reboot`); após o boot, a Tela Soma aparece.
- [ ] Com Tela Soma ativa, errar a soma, apagar e acender; novos números e sem desbloqueio.
- [ ] Mudar a chave por adb com a tela de bloqueio visível; o comportamento muda sem `pkill` (valida o Passo 2).

Para o relatório, capturar a tela com:

```
adb exec-out screencap -p > ~/tela_soma.png
```

Reverter e problemas encontrados

Ver o que foi alterado em cada projeto:

```
cd ~/aosp && repo status
cd ~/aosp/frameworks/base && git status --short && git log --oneline -3
cd ~/aosp/packages/apps/Settings && git status --short && git log --oneline
-3
```

Descartar alterações não commitadas de um projeto (irreversível; rodar o `git clean -fdn` antes para simular):

```
git reset --hard HEAD
git clean -fdn   # mostra o que seria apagado
git clean -fd    # apaga arquivos novos não rastreados
```

Voltar ao estado de um commit anterior: `git reset --hard <hash>` (hash obtido no `git log`).

Problemas encontrados e soluções:

Problema	Causa	Solução
Alterações da primeira tentativa não apareciam	Código não chegava ao emulador e faltavam pontos de integração	Recomeço do zero e validação do ciclo de build (Etapa 2)
Alteração em `system/core/.../android_filesystem_config.h`	Mudança desnecessária e arriscada da tentativa anterior	Revertida com `git reset --hard HEAD`
`'when' expression must be exhaustive ... 'Math'`	`when` do Kotlin sem o novo valor do enum	Ramo `Math -> Pin` no `AuthenticationRepository`
Mesmo erro após a correção	Arquivo não estava salvo no momento da build	Conferir com `grep` ou `git diff --stat` antes de compilar
`Device must be bootloader unlocked` / `Read-only file system`	Emulador iniciado sem `-writable-system`	Iniciar com `-writable-system`, `adb remount`, `adb reboot`, `adb remount`
`adb: don't know how to sync system_ext partition`	Terminal sem `envsetup` e `lunch`	Carregar o ambiente ou `export ANDROID_PRODUCT_OUT=~/aosp/out/target/product/emulator_x86_64`
Tela Soma some após reiniciar	Propriedades `debug.` não persistem	Migração para `Settings.Secure` (Etapa 7)
`git status` não lista arquivos novos	Configuração local esconde não rastreados	Conferir com `ls` ou `git status -u`